DSA DryRun

Interactive Code Execution Visualizer

Document type	Full project specification + roadmap
Purpose	Context document for AI coding assistants (Copilot / Claude Haiku)
Stack	React + Node.js + PostgreSQL + Redis + Docker
Target deadline	4 weeks to working MVP
Interview	Cisco SWE — 6 weeks from project start
Author	Project owner (you) + Claude Sonnet

How to use this document: Share this PDF with your AI coding assistant (GitHub Copilot, Claude Haiku in VS Code, or any LLM). It contains the full problem statement, chosen stack, system architecture, 4-week build plan, and all design decisions. The model can use it as complete context to help you implement each phase without repeated re-explanation.

01 — PROBLEM STATEMENT

What we are building — and why

Most DSA learners on platforms like LeetCode copy solutions without truly understanding the internal mechanics: how variables change at each step, how the call stack grows during recursion, why a pointer moves left vs right, what a DP table transition actually means. Traditional text explanations are static. DSA understanding is fundamentally dynamic and visual.

Core insight

There is a gap between 'I can read this solution' and 'I understand this algorithm'. This platform closes that gap by making code execution visible — every variable, every pointer, every stack frame, every state transition — animated in real time with AI explaining the 'why' at each step.

What users can do

- Select a DSA problem (Two Sum, Binary Search, Fibonacci, BFS/DFS, DP problems, etc.)
- Write or paste their solution in a Monaco code editor (same as VS Code)
- Run the code and step through it one line at a time
- Watch variable values update live in animated cards
- See pointer movement on array/tree/graph visualizations
- View the call stack grow and shrink during recursion
- Read AI-generated explanations per step: 'Why did left pointer move here?'
- Get time/space complexity analysis derived from the actual execution trace
- Test against multiple test cases with pass/fail per case

---

02 — SYSTEM ARCHITECTURE

Full system design

The system is designed as a set of logically separated services even if deployed initially as a monolith. Each layer has a clear responsibility boundary. This is intentional — the monolith-first approach means we feel the pain of coupling before splitting into microservices, which is how real production systems evolve.

Client layer	
React frontend	Monaco editor, visualization panels, step controls, variable state UI
Nginx	Reverse proxy, SSL termination, static file serving
API Gateway	Rate limiting, auth verification, request routing

Microservices layer	
Problems service	CRUD operations on problems, search, difficulty filtering
Auth service	JWT issuance, refresh token rotation, session management

Execution service	Receives submissions, enqueues jobs, returns jobId
AI explain service	Calls Claude/OpenAI per step, caches in Redis, streams response
Execution sandbox	
Job queue	Redis + BullMQ — decouples HTTP from execution, enables worker scaling
Worker + Docker	Spawns isolated container per run, enforces resource limits
Data layer	
PostgreSQL	Users, problems, submissions, test cases — relational source of truth
Redis	Job queue broker, session cache, AI response cache
Object storage	Test case files, boilerplate code, problem assets
Prometheus + Grafana	Metrics scraping, dashboards, alerting

Key architectural decisions and their rationale

Why a job queue for code execution?

If 100 users submit code simultaneously and the server runs each synchronously, it blocks. A queue (Redis + BullMQ) decouples the HTTP request from execution. POST /submit returns { jobId } immediately. Workers consume jobs independently. This is horizontal scaling: add more workers = more throughput, no code changes.

Why Docker for sandboxing?

Untrusted code can rm -rf /, read environment variables, open network connections, and consume infinite CPU. Docker uses Linux namespaces (process, network, filesystem isolation) and cgroups (CPU/memory limits). We run: docker run --memory=128m --cpus=0.5 --network=none --read-only --rm. Each flag is a specific OS-level isolation mechanism.

Why WebSockets for results?

HTTP request-response is pull-based. Execution takes 1-10 seconds. Polling every 500ms wastes 95% of requests. WebSockets give us a persistent TCP connection — the worker pushes each execution step as it happens. The client sees live updates without polling.

Why Redis for the cache AND the queue?

Redis serves two distinct roles here. As a queue broker it persists job state, supports retry logic, and enables distributed workers. As a cache it stores AI step explanations (same code + same step = cache hit, saving API cost). Both use cases benefit from Redis's in-memory speed and atomic operations.

Why monolith first?

Microservices add distributed systems complexity: network latency between services, distributed tracing, independent deployments, data consistency. Starting monolithic means we validate the product first. We split out the Execution service (different scaling needs: CPU-heavy) and Auth service (security boundary) only once the seams are clear.

03 — TECH STACK

Chosen technology stack with justification

Layer	Technology	Why chosen
Frontend	React + Vite	Component model suits visualization panels. Vite = fast HMR dev experience.
Code editor	Monaco Editor	Same engine as VS Code. Syntax highlighting, themes, line decorations built in.
State mgmt	Zustand	Lighter than Redux. Sufficient for execution step state + variable history.
Visualization	D3.js + custom SVG	D3 for data-driven array/graph animations. Custom SVG for call stack frames.
Backend	Node.js + Express	JS everywhere reduces context switching. Express is minimal and explicit.
Job queue	BullMQ + Redis	BullMQ is the modern BullJS successor. Named queues, retries, rate limiting built in.
Database	PostgreSQL	Relational model fits problems + submissions. ACID guarantees. pg driver, no ORM initially.
Cache/queue broker	Redis	Sub-millisecond reads. Native pub/sub for WebSocket fan-out. Atomic ops for queue.
Execution sandbox	Docker	OS-level isolation via namespaces + cgroups. Network disabled per run.
Code tracer	Python sys.settrace()	CPython hook that fires on every line, call, return. Captures full variable state.
Real-time	Socket.io	WebSocket with HTTP long-poll fallback. Room-based isolation per jobId.
AI layer	Anthropic Claude API	Step explanations + hint generation. Streamed via SSE to frontend.
Reverse proxy	Nginx	SSL termination, static file serving, upstream proxying to Node.
Containerization	Docker + Compose	Dev environment as code. Prod: Compose on VPS.
CI/CD	GitHub Actions	Lint, test, Docker build, push to GHCR, deploy on push to main.
Monitoring	Prometheus + Grafana	prom-client instruments Express. Grafana dashboards for latency, queue depth.
Hosting	DigitalOcean / Hetzner	~$6/mo VPS. SSH deploy. Real public URL for interview demo.

Project folder structure

This is the target structure at the end of Week 2. Phase 1 starts with only client/ and server/.

```
dsa-dryrun/
■■■ client/                    # React + Vite frontend
■   ■■■ src/
■   ■   ■■■ pages/              # ProblemList.jsx, ProblemDetail.jsx
■   ■   ■■■ components/         # Editor.jsx, VizPanel.jsx, StepControls.jsx
■   ■   ■■■ store/              # Zustand stores (executionStore, authStore)
■   ■   ■■■ hooks/              # useWebSocket.js, useExecution.js
■   ■■■ vite.config.js
■■■ server/                    # Express monolith (splits in Phase 4)
■   ■■■ routes/                 # problems.js, submissions.js, auth.js
■   ■■■ services/               # problemService.js, executionService.js
■   ■■■ workers/                # executionWorker.js (BullMQ consumer)
■   ■■■ sandbox/                # dockerRunner.js — spawns containers
■   ■■■ tracer/                 # pythonTracer.py — sys.settrace hook
■   ■■■ db/                     # postgres.js (pool), migrations/
■   ■■■ index.js
■■■ nginx/                     # nginx.conf, Dockerfile
■■■ monitoring/                # prometheus.yml, grafana dashboards
■■■ .github/workflows/         # ci.yml, deploy.yml
■■■ docker-compose.yml         # dev environment
■■■ docker-compose.prod.yml    # production overrides
■■■ .env.example               # all required env vars documented
```

docker-compose.yml — day 1 starting point

```yaml
version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: dsadryrun
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: dev
    ports: ['5432:5432']
    volumes: [postgres_data:/var/lib/postgresql/data]
  redis:
    image: redis:7-alpine
    ports: ['6379:6379']
  server:
    build: ./server
    ports: ['3001:3001']
    environment:
      DATABASE_URL: postgresql://dev:dev@postgres:5432/dsadryrun
      REDIS_URL: redis://redis:6379
    depends_on: [postgres, redis]
  client:
    build: ./client
    ports: ['5173:5173']
    depends_on: [server]
volumes:
  postgres_data:
```

05 — 4-WEEK ROADMAP

Build plan — 4 weeks to flagship MVP

Each week has a clear deliverable you can demo. The rule: never build features you can't demo by end of week. A working visualizer for 10 problems is worth more than a broken one for 100. Seed the DB with exactly these 10 problems: Two Sum, Binary Search, Valid Parentheses, Merge Sorted Arrays, Fibonacci, BFS, DFS, Dijkstra, LRU Cache, Climbing Stairs (DP).

WK 1 — **Foundation — full-stack skeleton + database**

Monolith, REST API, PostgreSQL, Docker dev env, Auth

Concepts learned: 3-layer architecture · REST API design · Connection pooling · JWT refresh tokens · Docker networking

WK 2 — **Code execution engine — the hard part**

Docker sandbox, BullMQ job queue, WebSockets, async arch

Concepts learned: Message queue (producer/consumer) · Container isolation (namespaces+cgroups) · WebSocket vs HTTP

WK 3 — **DSA visualizer + AI explanation layer**

Python tracer, variable viz, Claude API, Redis cache, complexity

Concepts learned: Instrumentation & tracing · Cache-aside pattern · API rate limiting · Streaming responses · Cost optimization

WK 4 — **Production-grade — CI/CD, monitoring, deploy**

GitHub Actions, Nginx SSL, Prometheus, Grafana, live URL

Concepts learned: CI/CD pipeline · TLS termination · Prometheus metrics · Observability · Zero-downtime deploy · Secret management

Daily schedule — Week 1 (detailed)

Day 1–2	Repo + dev environment	Create GitHub repo. npm create vite client. Express server scaffold. docker-compose with Postgre
Day 3–4	Problems API + database schema	CREATE TABLE problems (id, title, difficulty, description, starter_code). CREATE TABLE test_cas
Day 5–6	Frontend — problem list + Monaco editor	ProblemList page fetching from API. ProblemDetail with Monaco editor. Zustand store for current p
Day 7	Auth — JWT + refresh tokens	POST /auth/register, POST /auth/login. Access token (15min) + refresh token (7d). httpOnly cookie

06 — EXECUTION ENGINE DEEP-DIVE

How the code execution pipeline works

This is the most technically complex and interview-worthy part of the project. Understanding it deeply is essential — every component maps to a real system design concept.

Step-by-step execution flow

1	**User submits code**	POST /api/submit { code, language, problemId, testCases }
2	**Execution service enqueues**	Creates BullMQ job in 'code-execution' Redis queue. Returns { jobId } immediately — HTTP request is done.
3	**Client subscribes**	React opens WebSocket: socket.emit('subscribe', { jobId }). Socket.io joins room named jobId.
4	**Worker picks up job**	BullMQ worker (separate Node process) pulls next job from queue. Spawns Docker container.
5	**Docker sandbox runs**	docker run --rm --memory=128m --cpus=0.5 --network=none --read-only user-code:latest. Container has 5s timeout.
6	**Python tracer captures steps**	sys.settrace() hook fires on every line. Captures { line, locals, globals, call_stack } as JSON array.
7	**Worker emits steps**	For each trace step, worker emits to Socket.io room: socket.to(jobId).emit('step', stepData).
8	**Frontend updates**	React receives each step, animates variable cards, highlights current line in Monaco, updates viz panels.
9	**AI explanation requested**	Per step (throttled), client calls GET /api/explain?stepHash=xyz. Checks Redis cache first.
10	**Result stored**	Final pass/fail per test case written to submissions table. User can revisit any submission.

Python execution tracer — the core IP

```python
# server/tracer/tracer.py
import sys, json, copy

def trace_code(code_string):
    steps = []
    call_stack = []

    def tracer(frame, event, arg):
        if event == 'call':
            call_stack.append({
                'fn': frame.f_code.co_name,
                'args': dict(frame.f_locals)
            })
        elif event == 'line':
            steps.append({
                'line': frame.f_lineno,
                'locals': copy.deepcopy(frame.f_locals),
                'call_stack': copy.deepcopy(call_stack),
                'event': 'line'
            })
        elif event == 'return':
            if call_stack: call_stack.pop()
        return tracer

    sys.settrace(tracer)
    exec(compile(code_string, '<dsa>', 'exec'))
    sys.settrace(None)
    return json.dumps(steps)
```

CISCO INTERVIEW ANGLE

Docker uses Linux kernel namespaces (pid, net, mnt, uts, ipc) to isolate the container process, and cgroups to enforce CPU/memory limits. --network=none removes the network namespace entirely. This is OS-level isolation, not VM-level — Cisco engineers care deeply about this distinction. When asked 'how did you secure the code execution?', you can describe the exact kernel mechanisms.

07 — CISCO INTERVIEW STRATEGY

Using this project to ace the Cisco interview

Cisco is a networking and infrastructure company. They respect candidates who understand what happens underneath the abstractions — not just 'I used WebSockets' but 'I know it's a TCP connection with an HTTP upgrade handshake'. This project gives you real examples for every networking and systems concept they will probe.

Project-to-interview mapping

What you built	Networking concept	How to explain it to Cisco
WebSocket real-time results	TCP persistent connections	HTTP/1.1 Upgrade header, WS frame format, ping/pong keepalives, multiplexing v
Nginx reverse proxy	Load balancing + proxy	Layer 7 routing, upstream keepalives, connection pooling, health checks
TLS via Certbot	SSL/TLS handshake	Certificate chain, SNI, TLS 1.3 0-RTT, ALPN negotiation, cipher suites
Docker --network=none	Network namespaces	Linux netns, veth pairs, bridge networking, iptables rules Docker creates
Rate limiting on API	Flow control / QoS	Token bucket algorithm (same as network rate limiting), leaky bucket variant
Redis pub/sub for WebSocket	Message passing / multicast	Publisher-subscriber pattern, topic routing, analogous to multicast groups
Prometheus metrics scraping	SNMP / pull-based monitoring	Prometheus pull model vs push (SNMP trap), metric types: counter/gauge/histogra
BullMQ job queue	Packet queuing / QoS	FIFO queue, priority queues, head-of-line blocking, congestion analogy
Docker networking (bridge)	Virtual networking	Bridge = L2 switch, veth = virtual cable, iptables NAT for external access

Questions they will ask — your answers

Q: Design a scalable code execution service.

Describe exactly what you built: HTTP endpoint returns jobId, BullMQ queue decouples request from execution, Docker workers run sandboxed containers, WebSocket streams results. Then extend it: 'To scale to 10k concurrent users I'd run N worker replicas behind the same Redis queue — they auto-distribute. For multi-region I'd use a distributed message broker like Kafka instead of Redis.'

Q: How did you handle security in code execution?

Linux namespaces isolate process/network/filesystem. cgroups enforce CPU (0.5 core) and memory (128MB) limits. --read-only prevents filesystem writes. --network=none removes network namespace entirely. 5-second timeout kills runaway processes. Container is destroyed after each run (--rm). No host volume mounts.

Q: Explain your real-time architecture.

WebSocket over Socket.io. Client subscribes to a room named by jobId after submission. Worker emits step events to that room. This is push-based — server initiates, not client. I chose this over polling because polling at 500ms intervals for 5 seconds wastes 10 requests per submission. With WebSockets it's 0 wasted round-trips.

Q: How does your monitoring work?

prom-client instruments Express middleware — every route records request duration as a histogram, active WebSocket connections as a gauge, queue depth as a gauge polled from BullMQ. Prometheus scrapes /metrics every 15s. Grafana queries Prometheus. This is the pull model — opposite of SNMP traps which are push.

Q: What would you change if you had to handle 100x traffic?

Current bottlenecks: (1) Single Postgres — add read replicas. (2) Single Redis — Redis Cluster for queue partitioning. (3) Workers — horizontal scale (stateless, just add more). (4) Nginx — replace with a cloud load balancer (AWS ALB). (5) Code execution — switch to Firecracker microVMs (AWS Lambda's approach) for faster cold starts than Docker.

08 — WEEKS 5-6 PREP PLAN

After the build — interview prep weeks

Week 5 and 6 are not for building new features. They are for internalizing what you built, preparing system design answers, and drilling Cisco-specific networking depth. Your project is the case study for every system design question.

Week 5 — system design stories

- For every major component, practice a 2-minute explanation: what it does, why you chose it over the alternative, and what breaks first at scale.
- Practice the 'blank whiteboard' version: given only a prompt, draw the architecture you built and explain each connection.
- Write your architecture document (ARCHITECTURE.md). Forcing yourself to write it reveals gaps in your understanding.
- Practice failure mode analysis: 'What happens if Redis goes down? What if a Docker worker crashes mid-execution? What if Postgres is full?'
- Record yourself explaining the system. Watch it back. Any hesitation = gap to fill.

Week 6 — Cisco networking deep-dives

For each topic below, know it at protocol level — not just 'what it does' but 'how it works at the packet level'.

TCP/IP stack

3-way handshake, seq/ack numbers, sliding window, congestion control (CUBIC/BBR), TIME_WAIT state. You used TCP everywhere — WebSockets, Postgres connections, Redis connections.

HTTP/1.1 vs HTTP/2 vs HTTP/3

Pipelining, HOL blocking, HPACK compression, multiplexing, QUIC/UDP. Your Nginx config can specify HTTP/2 — understand what you're enabling.

DNS resolution

Recursive vs iterative resolution, TTL, CNAME vs A records, negative caching. Your services talk to each other by hostname inside Docker — that's internal DNS.

TLS 1.3

Handshake reduction to 1-RTT, ECDHE key exchange, session resumption. You terminate TLS at Nginx — know what happens in those first few milliseconds.

Load balancing algorithms

Round-robin, least connections, IP hash, consistent hashing. Nginx uses round-robin by default. Consistent hashing matters for sticky WebSocket sessions.

Linux networking

iptables, network namespaces, veth pairs, bridge devices, tc (traffic control). Docker creates all of these under the hood.

The 48-hour pre-interview checklist

- Ensure live URL is up and demo-able on your phone
- Have Grafana dashboard open in a browser tab
- Know exact Docker run command you use for sandboxing (memory to --read-only flags)
- Can draw system architecture from memory on a whiteboard in 3 minutes
- Can explain cache-aside pattern, producer/consumer pattern, and circuit breaker pattern

- Have GitHub repo link ready — README has architecture diagram and deployment instructions
- Practiced answering 'what would you do differently?' with honest tradeoffs
- Can explain what sys.settrace() does at the CPython interpreter level

CISCO INTERVIEW ANGLE

The single most powerful thing you can do in a Cisco interview is speak about your project at two levels simultaneously: the application level ('I built a code execution sandbox') AND the systems level ('which uses Linux network namespaces — the same mechanism Docker uses to isolate container networking'). That dual-level articulation is what separates senior engineers from junior ones in Cisco's evaluation rubric.

09 — CONTEXT FOR AI ASSISTANTS

Instructions for AI coding assistants reading this document

If you are an AI assistant (GitHub Copilot, Claude Haiku, GPT-4, or similar) reading this document as context, the following section is written directly for you.

Project identity

This is 'DSA DryRun' — an interactive DSA algorithm visualizer and step-by-step code execution platform. It is the developer's flagship portfolio project targeting a Cisco SWE interview. Every technical decision should reflect senior engineering judgment.

Stack — do not deviate

Frontend: React + Vite + Zustand + Monaco Editor + Socket.io client. Backend: Node.js + Express + PostgreSQL (pg pool, no ORM) + Redis + BullMQ + Socket.io. Tracer: Python sys.settrace(). Sandbox: Docker. Infra: Nginx + Docker Compose + GitHub Actions + Prometheus.

Current phase

Check with the developer which week/phase is active. Default assumption: Phase 1 (monolith, REST API, DB schema, auth). Do not suggest microservices refactoring until Phase 4.

Code style

No ORMs — raw SQL with parameterized queries. No magic frameworks. Explicit over implicit. Comments should explain WHY, not WHAT. Every function should do one thing.

When generating code

Always include error handling. Always use environment variables for config. Never hardcode credentials. Write code that a senior engineer would not be embarrassed to show in an interview.

Architecture decisions are final

Do not suggest replacing BullMQ with AWS SQS, Redis with Memcached, or Postgres with MongoDB. The stack was chosen deliberately. Suggest improvements within the chosen stack.

Interview context

This developer has a Cisco interview in 6 weeks. When explaining architectural decisions, always note the networking/systems concept it maps to (e.g., rate limiting = token bucket, WebSockets = TCP persistent connection). This framing is deliberate and should be maintained.

Document generated from a conversation between the project owner and Claude Sonnet 4.6. Contains: problem statement, system architecture, tech stack decisions, 4-week roadmap, execution engine design, Cisco interview strategy, and AI assistant context.